28. Logging: journald, syslog, and log files

28. Logging: journald, syslog, and log files

Phase: Phase 10 — Observability and troubleshooting
Estimated time: 1–2 days
Slug: `28-logs-rsyslog`

Learning objectives

- Find logs under /var/log
- Use journalctl effectively
- Rotate logs conceptually
- Correlate time across hosts

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

When something breaks at 03:00, the first question is "what did the system say just before it broke?" Logs are the system's own answer. On a modern Linux host two worlds overlap: **systemd-journald** (binary journal, queried with `journalctl`) and **syslog** (plain text files such as `/var/log/syslog` or `/var/log/messages`, written by rsyslog). You must be able to read both, filter fast, and know when logs are being lost or have filled the disk.

Skill goal: go from "the service is down" to the exact error line in under two minutes, without scrolling through a million lines.

Core concepts

Term	Meaning
journald	Service that collects stdout/stderr of units, kernel messages, and syslog() calls into a structured journal
journalctl	Query tool for the journal (filter by unit, time, priority, boot)
rsyslog	Classic syslog daemon; writes text files and can forward over the network
facility	Source category: kern, auth, daemon, user, local0-local7
priority	Severity 0-7: emerg, alert, crit, err, warning, notice, info, debug
persistent journal	Journal kept in `/var/log/journal` and surviving reboot
volatile journal	Journal kept in `/run/log/journal`, lost on reboot
logrotate	Periodic rename/compress/delete of text logs so they do not fill disk

```
app stdout/stderr --+
syslog() call ------+--> journald --+--> /var/log/journal (persistent)
kernel (dmesg) -----+               |--> /run/log/journal (volatile)
                                    +--> rsyslog --> /var/log/syslog, auth.log
                                                 \--> remote log server
```

The most important flag to memorize is `-u UNIT`: show only messages from one systemd unit. `journalctl -u nginx` beats `grep nginx` over everything.

Worked examples

Show the last 50 lines of one service, then follow live:

```
journalctl -u ssh -n 50 --no-pager
journalctl -u ssh -f
```

Only errors and worse since this boot:

```
journalctl -b -p err
```

Time window (absolute or relative):

```
journalctl --since "2026-10-21 09:00" --until "2026-10-21 09:30"
journalctl --since "-15 min" -u nginx
```

Previous boot (needs a persistent journal), useful after a crash or reboot:

```
journalctl --list-boots
journalctl -b -1 -p warning
```

Kernel messages only (like dmesg, but with timestamps and boot selection):

```
journalctl -k
dmesg --ctime | tail
```

Machine-readable output for scripts:

```
journalctl -u myapp -o json --no-pager | head -n 1
journalctl -u myapp -o cat       # message text only
```

Write a log line yourself and find it again:

```
logger -t labtest -p user.notice "hello from the lab"
journalctl -t labtest -n 3 --no-pager
grep labtest /var/log/syslog 2>/dev/null | tail -n 3
```

Check disk usage of the journal and trim it:

```
journalctl --disk-usage
sudo journalctl --vacuum-size=200M
sudo journalctl --vacuum-time=14d
```

Make the journal persistent if the directory is missing:

```
sudo mkdir -p /var/log/journal
sudo systemctl restart systemd-journald
```

Mini-lab

1. Run `journalctl -u ssh -n 20 --no-pager` (use `sshd` on RHEL-like hosts). Which fields appear on each line? Identify host, unit tag, and PID.
2. Run `logger -t lab28 "marker one"`; find it with `journalctl -t lab28`.
3. Run `journalctl -p err -b --no-pager | wc -l`. How many errors this boot?
4. Run `journalctl --disk-usage`. Is the journal persistent? Check with `ls /var/log/journal`.
5. Look at `/etc/logrotate.d/` and open one file. Identify rotate count, frequency, and the compress setting.
6. In a script, write a function that prints the flag you use to restrict `journalctl` to one unit. That is the exercise for this module.

Expected-style output of step 2:

```
Oct 21 10:02:11 host lab28[2314]: marker one
```

Decision guide / failure modes

Symptom	First move
One service misbehaves	`journalctl -u UNIT -n 100`
Host rebooted unexpectedly	`journalctl --list-boots`, then `-b -1 -p err`
No logs from before reboot	Journal is volatile; create `/var/log/journal`
Disk full under `/var/log`	`du -xh /var/log | sort -h | tail`, vacuum journal, fix rotation
Need logs from many hosts	Forward with rsyslog or a shipping agent
Auth problems	`journalctl -t sshd` or `/var/log/auth.log`

Failure modes to watch for:

- **Silent app**: app logs to a file you forgot about; systemd captures only stdout/stderr. Run it with `systemctl cat UNIT` and check `ExecStart`.
- **Rate limiting**: journald drops bursts ("Suppressed N messages"). A noisy loop can hide the real error.
- **Clock jumps**: wrong time makes `--since` windows miss events. Check `timedatectl`.
- **Permissions**: non-root users see only their own logs unless they are in the `systemd-journal` or `adm` group.
- **Unbounded logs**: a debug-level app fills the disk; set `SystemMaxUse=` in `/etc/systemd/journald.conf` and enable logrotate for text files.
- **Grep-only habit**: grepping text logs misses journal-only entries. Prefer `journalctl` filters first, grep second.

Common mistakes

- Deleting active log files to free space the wrong way
- No retention policy
- Ignoring permission on log dirs

Exercises

Work in `exercises/28-logs-rsyslog/`. Read that folder's `README.md` fully before coding.

```
cd exercises/28-logs-rsyslog
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `journal_unit_flag` so it prints the single `journalctl` option that restricts output to one systemd unit. This is a vocabulary drill: the function must print just the flag, with no extra text and no trailing spaces. Then run `./test.sh` until it

is green. After that, run the real command on a Linux host against a unit you know (for example ssh or cron) and paste three output lines into your notes.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/28-logs-rsyslog/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Where are traditional text logs?
2. journalctl -u meaning?
3. Why NTP for logs?

Further reading

- https://www.freedesktop.org/software/systemd/man/journalctl.html